

# タスクタイル 仕様書 V1

実績ベースのデッドライン管理アプリ

2026年7月21日 | 基本設計書V7準拠

## 1. 概要

- 名称:** タスクタイル（実績ベースのデッドライン管理アプリ）
- コンセプト:** 時間ではなく「実績（量）」でタスクを管理し、1日のノルマを自動逆算する成果特化型の生産性ツール。全体進捗ゲージは持たず、重要度順（レベルA→D）に並んだタスクタイルを上から消化させる。「今日何をどれだけやるか」を考えさせず、システムが提示したリストをただ潰していく状態を作り、過集中モードへの突入を支援する。
- 対象タスク:** 定量タスク（ページ数・問題数・回数など、数値で量を持つもの）
- 提供形態:** Web（ブラウザ） + Android アプリ。アカウント制のマルチユーザー。

## 2. 主な機能

### 2.1 逆算エンジンとタスク管理

- タスクはレベルA～Dに分類。各タスクは完全に独立した計算式を持ち、全体合算はしない。
- 算出されたノルマが優先度順のタイルとしてスタック表示される。
- 今日のノルマ** = 残量 ÷ 残り稼働日数。
- 二重デッドライン:** 実際の期日と、目標期日（=実際の期日 - マージン日数）。ノルマ計算は目標期日を優先。
- フレキシブル休日制:** 「週〇日稼働」で裁量を残す。休日の権利はタスク単位で管理。
- 自動休日消費:** 進捗が全く無かった日は休日の権利を1消費。1でも進めれば消費しない。

### 2.2 レベル別のシステム挙動

| レベル         | 位置づけ          | 挙動                               |
|-------------|---------------|----------------------------------|
| A (Must)    | 他者が関わる動かせない期日 | マージン厳しめ。安易なアーカイブ不可               |
| B (Should)  | 自分で決めた期日      | 標準マージン。期日再設定・アーカイブ可              |
| C (Want)    | 趣味・自己満        | マージンなし。ノルマは増やさず完了予定日を後ろ倒し（静かな通知） |
| D (Routine) | 終わりのないルーティン   | 逆算なし。固定量を毎日提示、翌日繰越なし             |

### 2.3 消化操作

- ワンタップ完了:** タイルをタップで今日のノルマを完了。
- 部分完了:** 長押しで実績数値を入力。タイルは残り「今日の残りノルマ」として各レベル最上部に留置。
- 完了の取り消し:** 完了済みタイルを再タップ → 確認の上で未完へ戻す（今日の実績を取り消し）。

### 2.4 日次リセットと徹夜対応

- 日付境界はデフォルト午前4時。
- 深夜0～4時の初回起動時のみ「延長しますか？」ポップアップ + 常時バナー表示。
- 延長の上限は翌日の正午までシステムで強制。
- サーバーはユーザーごとの「次回更新日時」を監視（全ユーザー一斉バッチではない）。

### 2.5 トリアージ（フェイルセーフ）

- ・ レベルA/Bで、計算上ノルマが標準ペースの1.5倍を超えると強制オーバーレイ。
- ・ 選択肢: マージン消費 / 休日返上 / 強行突破（枯渇時も非表示にせずグレーアウト表示）。
- ・ 最終分岐: レベルAは「関係者と合意済み」入力（フリクション）付きの期日再設定、期日超過でゾンビモード（逆算停止・無機質表示）。レベルBは期日再設定・アーカイブ。

## 2.6 認証・マルチユーザー

- ・ トークン認証（登録・ログイン・ログアウト）。Authorization: Token <key>。
- ・ ユーザーごとにデータ完全分離（他人のタスクは参照・操作不可）。
- ・ パスワードはハッシュ保存＋登録時に強度チェック（8文字以上等）。

## 3. 技術構成

- ・ **バックエンド**: Python / Django 5.2 (JSON API)
- ・ **フロントエンド**: Flutter (Android / Web の同一コード)
- ・ **データベース**: PostgreSQL (本番) / SQLite (開発)
- ・ **配信構成**: 同一オリジン。DjangoがビルドしたFlutter Web (webapp/) をwhitenoiseでルート配信し、/api/ はAPI。CORS不要。
- ・ **認証方式**: Cookieを使わないヘッダトークン (CSRF非該当)。

## 4. API仕様

認証以外の /api/ はすべて Authorization: Token <key> が必須。

| メソッド     | パス                          | 内容                   |
|----------|-----------------------------|----------------------|
| POST     | /api/auth/register/         | 新規登録（トークン発行）         |
| POST     | /api/auth/login/            | ログイン（トークン発行）         |
| POST     | /api/auth/logout/           | ログアウト（提示トークン失効）      |
| GET/POST | /api/tasks/                 | タイルスタック取得 / タスク作成    |
| POST     | /api/tasks/{id}/complete/   | ワンタップ完了              |
| POST     | /api/tasks/{id}/uncomplete/ | 完了の取り消し              |
| POST     | /api/tasks/{id}/progress/   | 部分完了（実績数値）           |
| GET/POST | /api/tasks/{id}/triage/     | トリアージ状態 / 選択適用       |
| POST     | /api/reset/extend/          | 更新時間の延長（上限=翌日正午）     |
| POST     | /api/reset/run/             | 次回更新日時の超過チェック & 日次処理 |

## 5. デプロイ構成

- ・ **ホスティング**: Render（無料枠）、render.yaml によるBlueprintデプロイ。
- ・ **設定**: 環境変数駆動（SECRET\_KEY / DEBUG / ALLOWED\_HOSTS / DATABASE\_URL / SECURE / CORS）。
- ・ **セキュリティ**: 本番はHTTPS強制・HSTS・セキュアCookie。gunicornで配信。
- ・ **日次リセット**: python manage.py run\_resets をcronで定期実行（起動時にも自動判定）。

## 6. 現時点の制限・今後の課題（V2候補）

- ・ タスクの編集・一般削除・完了/履歴一覧が未実装。
- ・ パスワードリセット（要メール基盤）・ログイン試行の回数制限が未実装。
- ・ 定性タスクのLLM自動分割（設計書 § 2の別フェーズ）は未実装。
- ・ iOS未対応（Android / Web のみ）。
- ・ 無料DBは約30日で期限切れのため、永続運用には Neon 等への移行が必要。
- ・ Web版更新時はService Workerキャッシュにより、既存利用者に旧版が一時的に表示される。